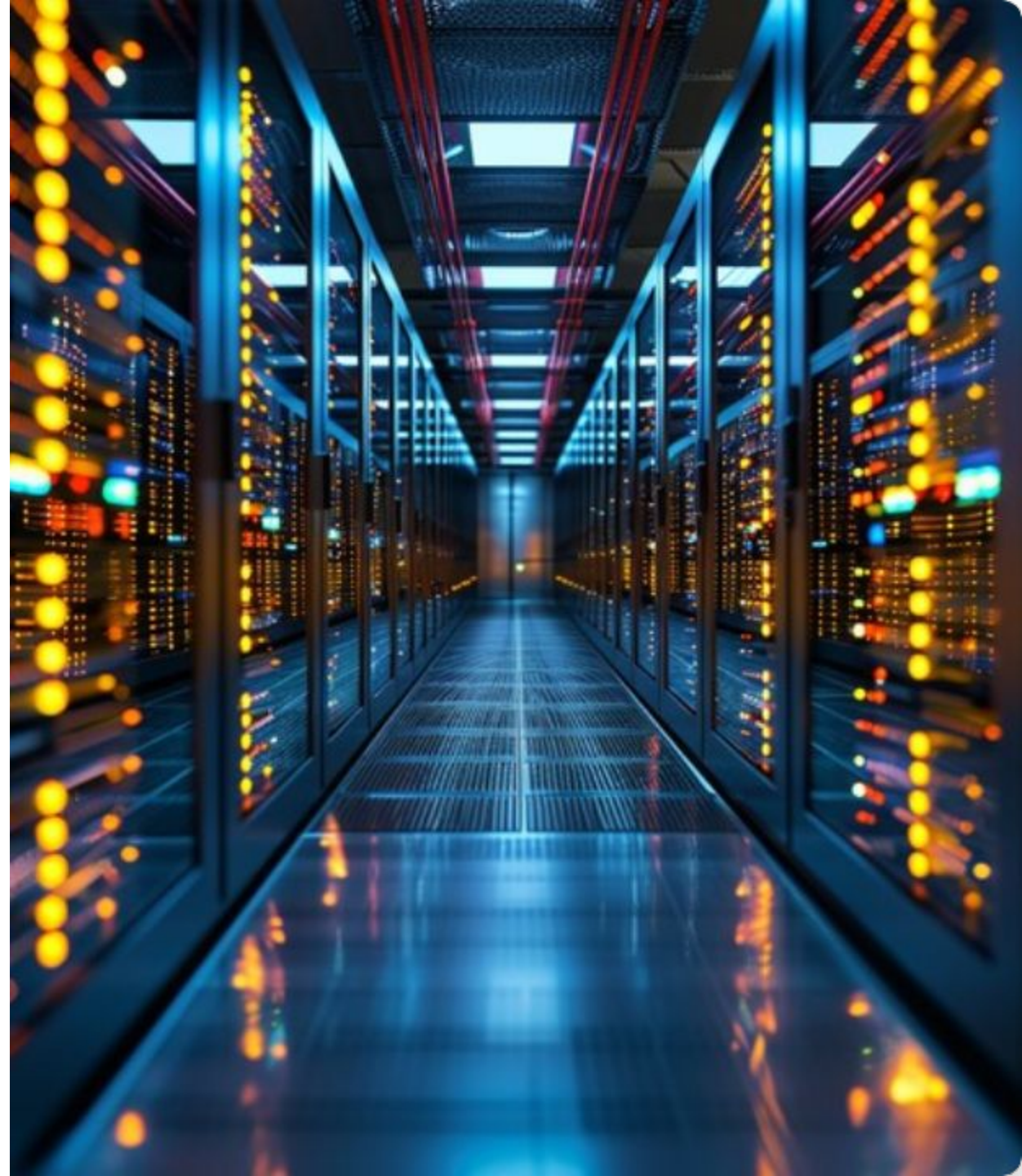


Arquitetura Moderna, Integração e Nuvem

Decisões de engenharia, resiliência em sistemas distribuídos e práticas reais de produção
para a formação de desenvolvedores.

**Monolito é legado e
microserviço é o novo?**



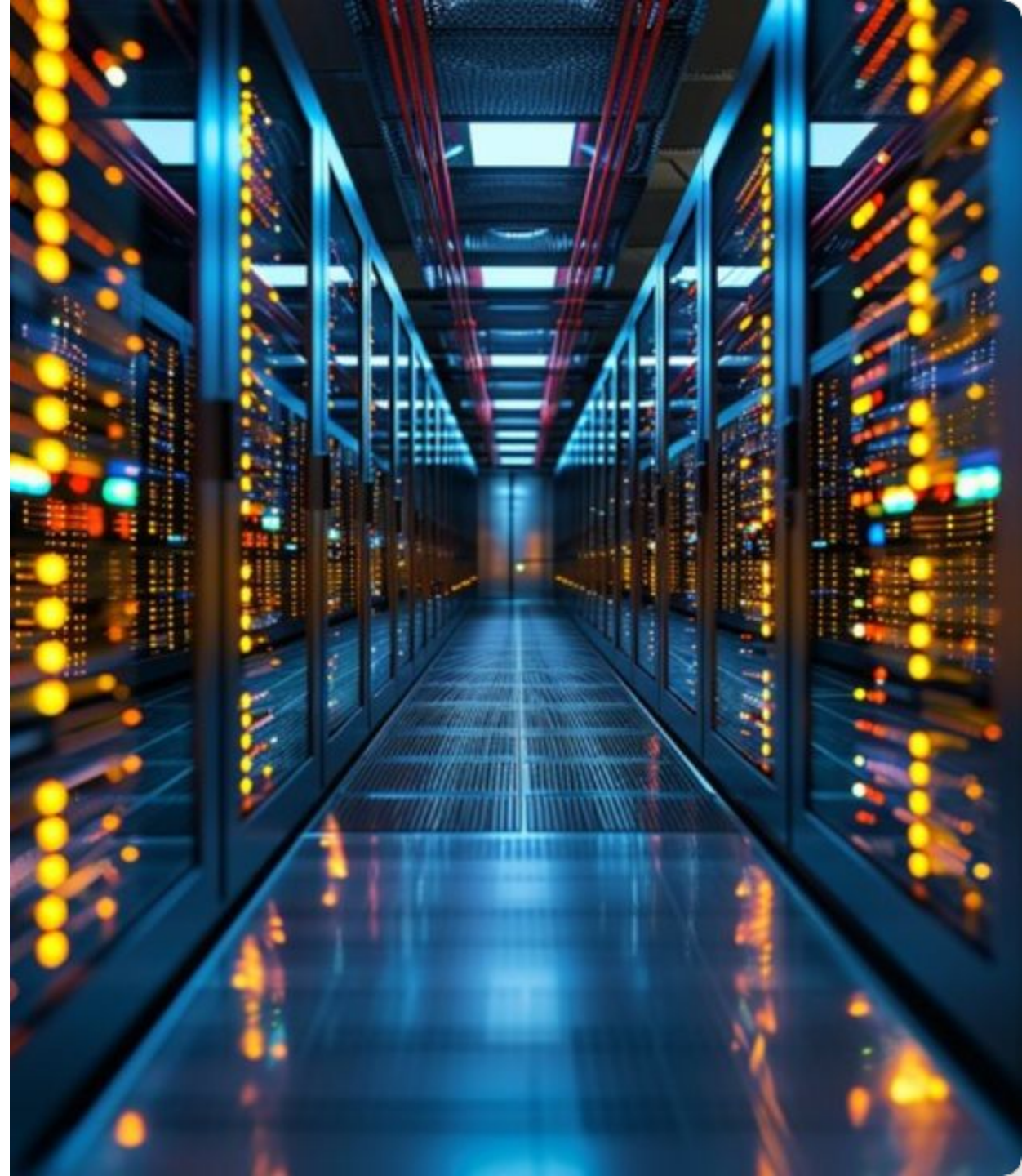
“Organizações que projetam sistemas são obrigadas a produzir projetos que são cópias das estruturas de comunicação dessas organizações”

Mas na prática é um grande depende

Além do Hype dos Microserviços

Decisões Baseadas em Trade-offs

A transição de monolitos para microserviços não é uma simples evolução natural, mas uma troca deliberada entre complexidade de código e complexidade operacional.



Monolito Modular vs Microserviços

Monolito Modular

- Fronteiras de domínio bem demarcadas no mesmo executável.
- Transações ACID nativas sem problemas de concorrência remota.
- Deploy atômico e depuração local simplificada
- Zero latência de rede na comunicação entre submódulos.

Microserviços Reais

- Velocidade de entrega a total rigor e dependência do contexto corporativo
- Escalabilidade granular de componentes críticos e isolados.
- Custo operacional elevado: governança, telemetria e CI/CD.
- Consistência eventual e necessidade obrigatória de resiliência.

Desenhando arquiteturas

Comunicação e Resiliência

Como projetar interfaces confiáveis, desacopladas e tolerantes a falhas em arquiteturas distribuídas de larga escala.

APIs de Alta Confiabilidade

- **Idempotência estrita:** Uso de chaves idempotentes em POSTs para prevenir cobranças duplicadas em falhas transitórias.
- **Paginação por cursor:** Substituição de offsets por cursores estáveis em coleções com milhões de registros.
- **Circuit Breakers:** Interrupção automática de chamadas a serviços instáveis para estancar o efeito dominó.
- **Backoff Exponencial com Jitter:** Retentativas que evitam sobrecarga síncrona nos bancos de dados.



Estratégias de Mensageria



Filas de Trabalho

RabbitMQ e SQS. Foco em consumo competitivo destrutivo, ideal para processamento de tarefas assíncronas e envio de notificações.



Streams de Eventos

Apache Kafka e Kinesis. Log imutável particionado com histórico gravado em disco, garantindo ordenação e replay temporal de eventos.



Padrão Saga

Coordenação de transações de negócio distribuídas (coreografada ou orquestrada) com ações compensatórias explícitas em caso de falha.

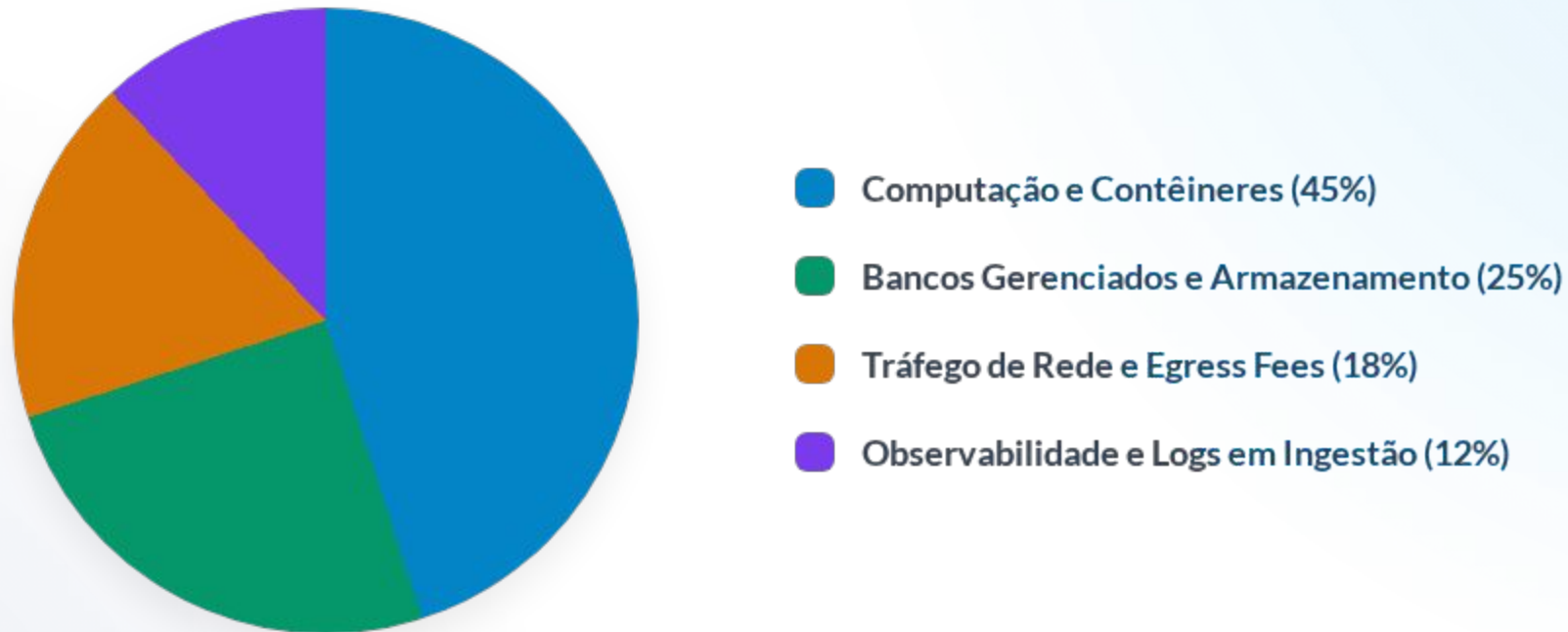
Padrões de Integração em Produção

Protocolo / Padrão	Modelo de Comunicação	Melhor Caso de Uso	Impacto Operacional
REST / JSON	Síncrono (Request / Response)	APIs públicas e consumo web mobile	Baixo (Universalmente suportado)
gRPC / Protobuf	Síncrono / Streaming binário	Comunicação interna de microsserviços	Médio (Contratos rígidos e HTTP/2)
Message Queues (SQS)	Assíncrono (Ponto a Ponto / PubSub)	Desacoplamento temporal de tarefas	Baixo a Médio (Dead Letter Queues)
Event Logs (Kafka)	Assíncrono (Event Streaming)	Pipelines analíticos e auditoria reativa	Alto (Gestão de brokers e offsets)

Nuvem, Serverless & Observabilidade

Economia em cloud, abstrações eficientes de execução e instrumentação em ambientes de missão crítica.

Composição de Custos em Cloud



A cultura FinOps demonstra que custos ocultos de tráfego entre zonas e armazenamento de logs frequentemente superam previsões iniciais de infraestrutura.

Os Três Pilares da Observabilidade

Além do Monitoramento Passivo

Sistemas modernos combinam métricas agregadas (RED method), logs estruturados com contexto serializado em JSON e distributed tracing com OpenTelemetry.

O rastreamento fim a fim permite diagnosticar gargalos e falhas de serviços antes que usuários finais sejam impactados em produção.



Perguntas e debate